

COMP3600 LAB1

Understanding Pyamaze

`Pyamaze` is a Python module that simplifies **maze generation and navigation**. It eliminates the need to manually code a **GUI** or use **Object-Oriented Programming**, making it easy to experiment with mazes and search algorithms.

🔷 What Pyamaze Does:

✔️ Creates **customizable mazes** effortlessly.

✔️ Includes **built-in tools** for visualization and manipulation.

✔️ Enables **easy pathfinding experiments** for AI search.

🔷 Key Features:

✔️ Uses a **grid-based system** (cells & walls).

✔️ Represents positions as **(row, column) coordinates**.

✔️ Supports **built-in movement functions** for agents.

✅ **No extra installation required** – Pyamaze uses Python's built-in **Tkinter** for GUI! 🚀

Installing Pymaze

open your VS Code terminal and run the following command:

```
pip install pyamaze
```

✅ This will install **Pyamaze** and all necessary dependencies.

Creating a Basic Maze

Use the following code to generate and display a **5×5 maze**:

```
from pyamaze import maze

m = maze(5, 5)  # Create a 5×5 maze
m.CreateMaze()  # Generate the maze
m.run()  # Display the maze
```

✅A **Tkinter window** will pop up, showing the maze you've built! 🏁



Essential Methods Documentation

Use these methods to create, visualize, and experiment with different maze structures before applying AI search algorithms! 🚀

Method	Description
`m = maze(rows, cols)`	Creates a maze with the specified number of rows and columns.
`m.CreateMaze(x,y)`	Generates a maze with the goal cell set at (x, y). If not specified, the goal defaults to the Up-right corner. Other parameters : `pattern` → Defines the maze style: 'v' (long vertical paths) or 'w' (long horizontal paths). `loopPercent` → Controls the number of loops in the maze. A higher value means more loops. `saveMaze` → If True, saves the maze structure to a CSV file for reuse. If False, generates a new maze each time. `loadMaze` → Loads a previously saved maze from a CSV file to recreate the same structure. `theme` → Sets the maze color theme. Requires `from pyamaze import COLOR` . Available options:

	`COLOR.light` , `COLOR.red` , `COLOR.cyan` , `COLOR.green` , `COLOR.yellow` .. etc
`m.run()`	Runs the Tkinter window to display the generated maze.
`a = Agent(m, x, y)`	Creates an agent at position (x, y) within maze m. If not specified, the default starting position is the bottom-right corner. you must import the Agent class, which represents a moving entity within the maze `from pyamaze import Agent` Other parameters : `shape` → Defines the agent's appearance: 'square' (default) or 'arrow'. `goal` → Sets a new goal position for the agent instead of the default maze goal. `filled` →Boolean (True/False): If True, the agent's shape fully occupies the cell. `footprints` → Boolean (True/False): If True, the agent leaves a trail while moving. Useful for path visualization. `color` → Defines the agent's color. Requires `from pyamaze import COLOR` .
`a.position(x,y)`	Changes the position of the agent after creation
`m.grid`	shows all the cells (x,y) of the maze
`m.maze_map`	Shows specific details of the opened and closed paths of the maze
`m.tracePath({a: m.path})`	Animates the **agent's movement** along a given path in real-time. Expects a **dictionary** where the **key** is the agent, and the **value** is the maze path. Other parameters : `delay` → Integer value that specifies the **time delay** (in milliseconds) between each movement step. `kill` → Boolean (`True/False`): If `True` , the **path disappears** after reaching the goal.

`m.enableArrowKey(a)`	Enables **keyboard arrow keys** to manually move the agent **a** inside the maze. Great for interactive exploration.
`m.enableWASD(a)`	Enables the **W, A, S, D** keys to manually move the agent **a** inside the maze.

💡To **learn more** and **discover other methods**, check out this incredible GitHub repository:

🔗 https://github.com/MAN1986/pyamaze.git

Quick Example: Maze with Pyamaze

Run the code, explore the maze, and see how it all works!”

```
from pyamaze import maze, agent, COLOR

# Step 1: Create a 7×7 Maze with custom properties
m = maze(10, 10)
m.CreateMaze(x=1, y=1, pattern='v', loopPercent=50, saveMaze=True, theme=COLOR.light)

# Step 2: Create an Agent at the starting position (bottom-right)
a = agent(m, x=m.rows, y=m.cols, shape='arrow', footprints=True, color=COLOR.green)

# Step 3: Trace the path with animation
m.tracePath({a: m.path}, delay=150, kill=False)

# Step 4: Run the Maze visualization
m.run()
```

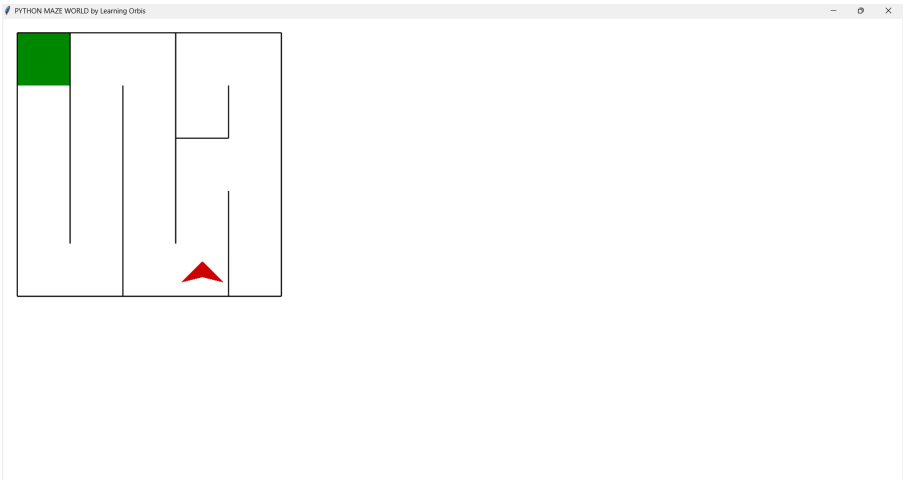
Tasks - Dive Into Pyamaze!

🟢 Exercise 1 (Easy)

Description:

Create a simple 5×5 maze and experiment with different maze patterns ('v' or 'w'). Place an agent at the any position far from the goal and display the maze. Change the agent’s shape and color to explore how it affects the visualization. Try to customize the maze as much as you can!

You can create something like:

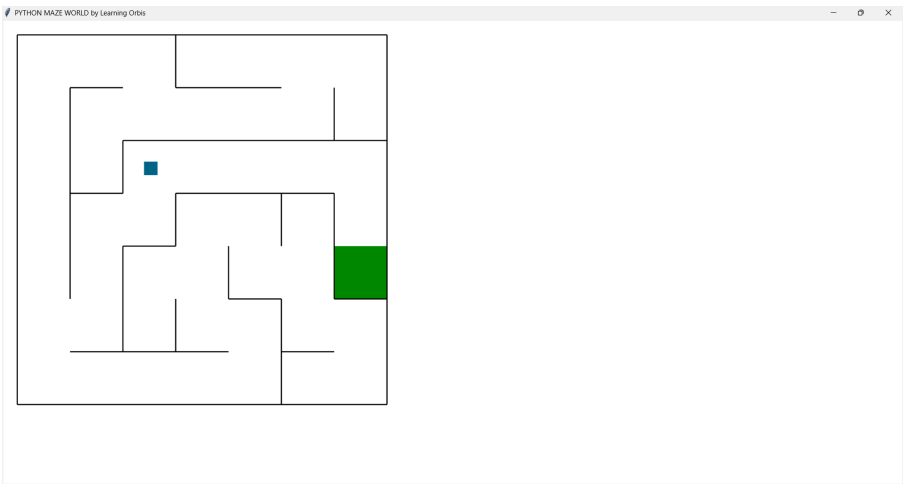


🔵 Exercise 2 (Medium)

Description:

Generate a 7×7 maze and set a goal for your agent at (5,7). Use Pyamaze's built-in pathfinding to calculate the shortest route to the goal `m.tracePath({a: m.path})`. Animate the agent's movement and experiment with the delay setting to move slower or faster.

should look like that with the Agent moving :



🟠 Exercise 3 (Challenging)

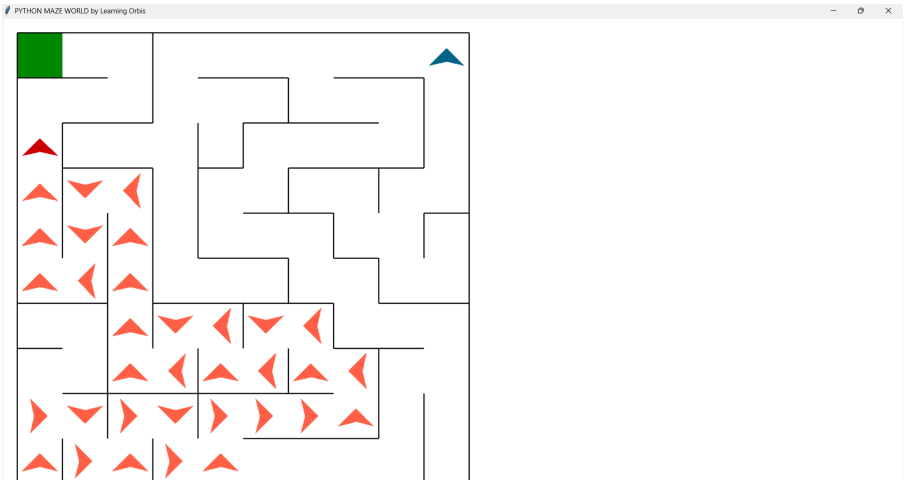
Description:

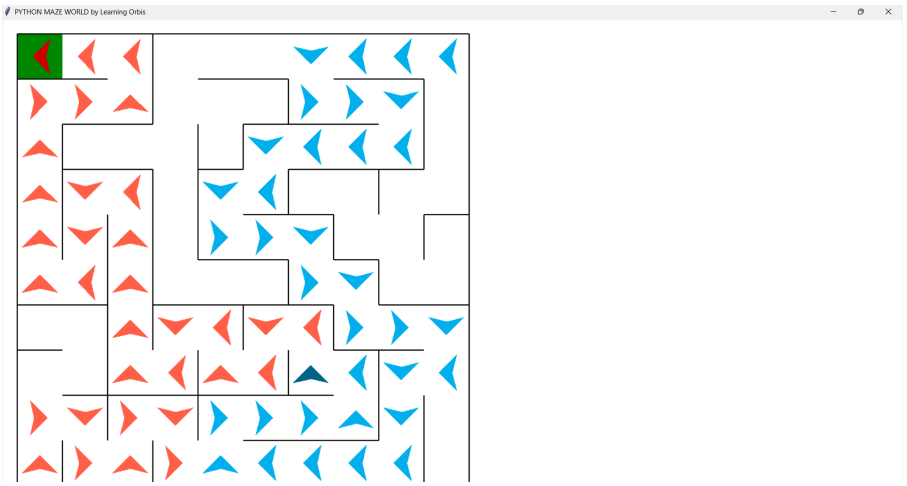
Create a **10×10 maze** with **two starting points** and **two goal points**. Place two agents:

- A **red agent** starting from the **bottom-left** corner.
- A **blue agent** starting from the **top-right** corner.

Both agents should have **arrow shapes** and **leave footprints** as they move. Use `m.tracePath({a: m.path})` to **visualize each agent's journey** as they navigate towards their respective goals. Experiment with different speeds by adjusting the `delay` parameter.

should look like this with the two Agents moving (one after the other) :





🔴 Exercise 4 (Advanced & Fun)

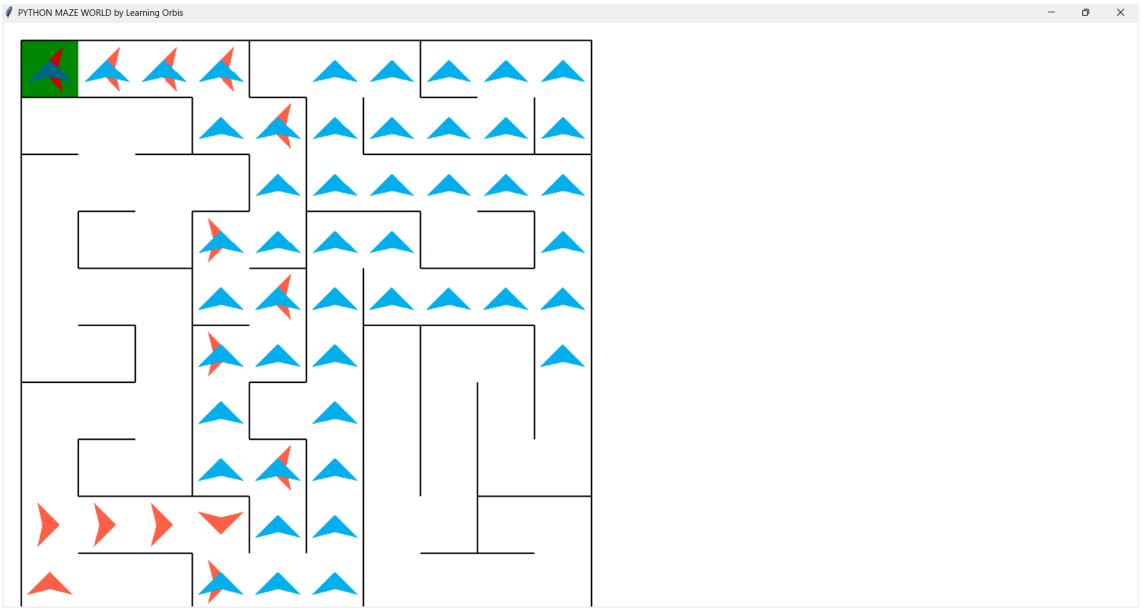
Description:

Compete against an AI-controlled agent and a user-controlled agent in a **10×10 maze**. The **AI agent** uses the built-in `m.tracePath({a: m.path})` function to navigate the maze, while you control your agent using the **arrow keys** `m.enableArrowKey(a)`.

Track how quickly both agents reach the goal. Experiment with different AI delay values and see at which **delay setting** your agent beats the AI! Ready for the challenge? It's time to race through the maze!

This is an example of me racing with the AI red agent, and AI won over me!

you think you can beat the AI? :))



✨ Done with Passion

Sultan Qaboos University - COMP3600

Dr. Noushath Shaffi

TA: Malak Al Bahri -137615-